

Deep reinforcement learning for the computation offloading in MIMO-based Edge Computing

Abdeladim Sadiki^a, Jamal Bentahar^{a,c,*}, Rachida Dssouli^a, Abdeslam En-Nouaary^b, Hadi Otrok^c

^a Concordia Institute for Information Systems Engineering, Concordia University, Montreal, Canada

^b Institut National des Postes et Telecommunications, STRS Lab, Rabat, Morocco

^c Department of EECS, Khalifa University, Abu Dhabi, UAE

ARTICLE INFO

Keywords:

Multi-access Edge Computing
Massive multiple-input multiple-output
Deep reinforcement learning
Computation offloading

ABSTRACT

Multi-access Edge Computing (MEC) has recently emerged as a potential technology to serve the needs of mobile devices (MDs) in 5G and 6G cellular networks. By offloading tasks to high-performance servers installed at the edge of the wireless networks, resource-limited MDs can cope with the proliferation of the recent computationally-intensive applications. In this paper, we study the computation offloading problem in a massive multiple-input multiple-output (MIMO)-based MEC system where the base stations are equipped with a large number of antennas. Our objective is to minimize the power consumption and offloading delay at the MDs under the stochastic system environment. To this end, we introduce new formulation of the problem as a Markov Decision Process (MDP) and propose two Deep Reinforcement Learning (DRL) algorithms to learn the optimal offloading policy without any prior knowledge of the environment dynamics. First, a Deep Q-Network (DQN)-based algorithm to solve the curse of the state space explosion is defined. Then, a more general Proximal Policy Optimization (PPO)-based algorithm to solve the problem of discrete action space is introduced. Simulation results show that our DRL-based solutions outperform the state-of-the-art algorithms. Moreover, our PPO algorithm exhibits stable performance and efficient offloading results compared to the benchmarks DQN and Double DQN (DDQN) strategies.

1. Introduction

Over the last years, with the great progress in the development of smart Mobile Devices (MDs) and wireless communication systems, there has been an explosive growth in the number of innovative applications that need a large amount of computing resources, such as ultra high definition media streaming, Virtual-Reality/Augmented-Reality (VR/AR) applications, real-time online 3D gaming, image processing, face recognition [1], some of which may also have a delay-sensitive characteristic [2]. However, even with their high performing modern hardware, MDs are usually limited in terms of computation capabilities, battery duration, and storage capacities. As a result, the Quality-of-Service (QoS) and Quality-of-Experience (QoE) of these applications are significantly impacted when tasks are executed on MDs [3,4].

To overcome this limitation, a viable solution was to leverage the powerful resources of specialized remote cloud servers to carry out the computation-intensive tasks. This approach is known as Mobile Cloud Computing (MCC) [5]. Although MCC could improve the battery life of MDs as well as the application experience [6], its major drawback is the long distance between the MDs and the cloud server, which results

in a significant network congestion, a substantial latency of service and performance degradation [2,7].

To address the above issues, a recent promising approach, known as Multi-access Edge Computing (MEC), has been proposed. The key idea underlying MEC is to push cloud-like computing capabilities to the network edge (e.g., base station) providing high performance services in close proximity to MDs [5,8,9]. Thereby, users can offload their computationally intensive and time-critical tasks to the nearby MEC server (to which they are connected through wireless connections) for processing rather than a centralized remote cloud server [10]. As a result, the potential congestion can be decreased, the service delay can be reduced and the energy usage of MDs can be significantly enhanced [11].

Currently, the 6th generation (6G) wireless communication network is getting great attention from the research community. Along with MEC, which will be one of the major elements of the 6G era [12], massive Multiple-Input Multiple-Output (MIMO) is a key enabler technology to deliver the needs of the next generation networks [13]. Massive MIMO involves deploying a large number of antennas at the

* Corresponding author at: Concordia Institute for Information Systems Engineering, Concordia University, Montreal, Canada.

E-mail address: bentahar@ciise.concordia.ca (J. Bentahar).

<https://doi.org/10.1016/j.adhoc.2022.103080>

Received 2 August 2022; Received in revised form 12 December 2022; Accepted 30 December 2022

Available online 5 January 2023

1570-8705/© 2023 Elsevier B.V. All rights reserved.

base stations, which leads to a significant improvement in the system's efficiency and throughput [14,15]. As a result, using massive MIMO is significant as it substantially assists computation offloading in MEC. In fact, by providing high spectral and energy efficiencies, massive MIMO will benefit the computation offloading with high transmission rates and lower energy consumption [16].

Although MEC has enormous benefits, there remain several challenges. To begin with, real-time applications are highly sensitive to the latency requirement, while service latency is affected by various elements in offloading [17], such as transmission and power allocation [10]. Moreover, the computation tasks are generated dynamically in the MDs, so deciding what part of the given task should be offloaded to the edge is a challenge in the offloading decision making process [7]. MDs are also limited in terms of energy consumption that needs to be considered. Furthermore, the MEC environment keeps changing over time and enjoys a stochastic characteristic at three levels: wireless channel, physical location of MDs and user mobility [7,10]. As a result, an efficient offloading framework under the stochastic MEC conditions has become a major challenge. A number of offloading schemes have been proposed in the literature [1,6,18,19]. The objective functions were designed as reducing the energy consumption, satisfying the latency requirement or finding a trade-off between the energy and latency [20]. The first studies were relying on classical optimization algorithms to solve these objective functions. However, they do not consider the challenging dynamic aspect of the MEC environment as they focus solely on performance of a quasi-static system [20]. The emerging Reinforcement Learning (RL) has been recognized as an effective approach to overcome the issue of dynamic settings in various applications [20–23]. Some of the existing proposals use Q-learning [5,20] to develop dynamic computation offloading methods without any prior knowledge of the MEC dynamics. Yet, due to the curse of state space explosion [24], this approach is quickly outdated in favor of Deep Reinforcement Learning (DRL), especially the Deep Q-Network (DQN) algorithm which is being used in the majority of the most recent solutions [1,25]. However, the common limitation in these proposals is that they are always considering discrete action space-based policies, which makes them powerless to control continuous quantities such as the energy or the latency. This issue is more challenging when massive MIMO is considered because of the additional complexity of the channel model and the possibility of simultaneous offloading [16].

In this paper, we study the computation offloading problem in a MEC server powered by massive MIMO technology. We consider the stochastic wireless channel variations and the dynamically generated tasks at each mobile user, and we propose novel DRL-based strategies that can learn a dynamic offloading policy under the varying MEC conditions. In addition to the DQN-based strategy, we propose a novel offloading strategy based on the Proximal Policy Optimization (PPO) technique to support continuous action space. It is worth mentioning that we are not simply using “off the shelf” DRL solutions, but we introduce a novel state and action spaces formulation of the problem that considers various aspects in a way that makes our PPO strategy highly efficient. Therefore, the main contributions of this work are summarized as follows:

- We formalize the computation offloading in a MEC with MIMO network under stochastic wireless conditions and task arrivals as a joint minimization problem between power consumption and offloading delay at each MD.
- To solve the formulated problem, we design a Markov Decision Process (MDP) where the state space, action space and reward function are carefully formalized.
- A DQN strategy is implemented based on our formulated MDP to learn a computation offloading policy that can minimize the system cost.
- A novel PPO strategy is introduced to derive better offloading policy over the continuous power allocation action space.

- A series of simulations are conducted to compare the performance of the proposed DRL-strategies. The results show that our PPO strategy gives better performance over the DQN strategy that is also used as benchmark. We also compare our strategies with the DDQN solution proposed in [3]. The results reveal that our DRL-based strategies outperform the state-of-the-art schemes.

The rest of this paper is organized as follows. Section 2 presents a review of the related work. Section 3 describes the system model and problem formulation. Section 4 proposes the DRL-based offloading model. Specifically the MDP formulation, the DQN and PPO-based strategies. Section 5 provides experiments and simulation results. Finally, Section 6 concludes the paper.

2. Related work

MEC is emerging as a potential technology to cope with the limitations of the MDs with regard to the rapid progress of the next generation of mobile applications [26]. Offloading execution to the edge improves the user experience by enhancing the device performance and lowering the energy consumption. Computation offloading techniques for MEC have lately been extensively studied. In [27], the authors proposed a partial offloading scheme as a joint minimization problem between the energy consumption of the smart mobile device and the latency of application execution using the dynamic voltage scaling technology. A novel framework was developed in [28] for offloading computation tasks from a MD to the edge using the Radio Network Information Service (RNIS) application programming interface (API) to drive the user offloading decision. Another approach for the computation offloading problem was given in [29]. In this work, the authors demonstrated that obtaining an optimal solution for the computation offloading problem is NP-hard, so they used a game theoretic approach to design an efficient offloading algorithm that can achieve the Nash equilibrium. Similarly, to reduce the delay and save the battery life of the user's devices, the authors in [30] proposed an efficient offloading algorithm by transforming the formulated NP-hard mixed integer non-linear problem into two solvable sub-problems, namely task placement and resource allocation. In [31], the authors studied the offloading problem over the 5th generation mobile network (5G). They created an intelligent offloading model based on a metric that can satisfy the latency requirements of user applications and achieve maximum energy saving.

Most of the previous mentioned proposals focus on the basic case in which both the users and the base station are equipped with a single antenna. This fails to leverage the benefits brought by the massive MIMO technology to MEC in terms of offloading performance [32]. For this reason, researchers begin to investigate MEC assisted with the MIMO technology. In [16], the authors studied the application of massive MIMO on MEC. They showed that MIMO boosts the performance of offloading in MEC by providing huge gains in spectral and energy efficiencies. Their results revealed that using more antennas reduce the energy consumption and the system delay. The work in [33] investigated the MEC computation offloading problem in massive MIMO enabled heterogeneous networks (HetNets). The formulated problem was defined as minimizing the energy consumption under a maximum latency requirement. An iterative low-complexity algorithm based on alternating optimization was proposed to solve this non-convex problem. Furthermore, the authors in [34] addressed the computation offloading problem in MEC with multi-user MIMO communication as a joint minimization of the energy consumption and time delay of MDs. The formulated mixed-integer non-linear programming problem is solved by developing offloading decisions based on semi-definite relaxation and rounding methods. Similar to [16], the simulation results revealed that the use of MIMO communication in MEC reduces sufficiently the energy consumption and time delay during computation offloading.

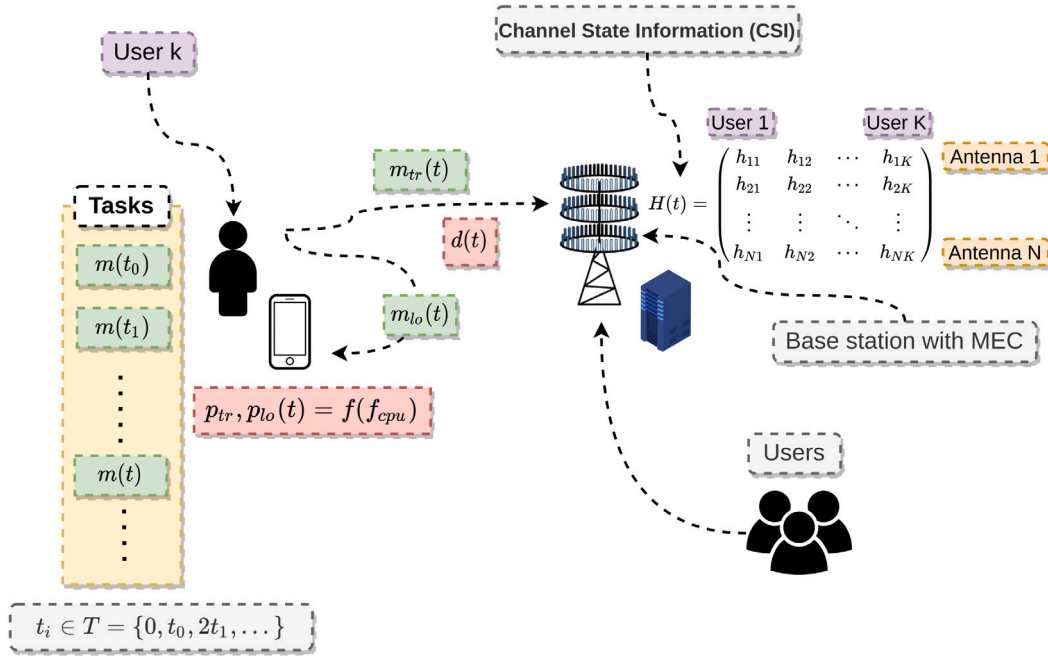


Fig. 1. Problem description.

Nevertheless, the MEC environment dynamics is usually complex and very challenging due to several reasons such as the stochastic network channel conditions, tasks arrival distribution, and power constraints of the MDs [24]. This makes the aforementioned classical optimization techniques very limited as they are mainly modeled based on a network snapshot and they must be reformulated when the dynamics changes over time. Besides, most of them need a high number of iterations and may provide a local rather than global optimum [10]. In fact, they mainly use heuristics to yield feasible solutions, which makes them subject to two major limitations [35–37]:

1. The divergence likelihood of the heuristic algorithms increases with the problem's input size.
2. The heuristic-based solutions do not allow the model to account for the dynamic changes of the environment and learn from previous experiences.

Fortunately, the emerging artificial intelligence technology has shown potential efficacy to tackle those limitations. Especially, RL is being identified as a suitable framework to deal with the stochastic MEC network environment. In [5], the authors used the Q-learning algorithm to jointly optimize the offloading decision with resource allocation while minimizing energy consumption on the MDs under the latency constraint. The same approach was adopted in [20]. The authors used the Q-learning algorithm to find the optimal policy for resource allocation and computation offloading in MEC. Their results show that the proposed solution leads to a significant decrease in energy consumption of the user's devices compared to baseline methods. However, classical RL algorithms cannot scale when the state space and action space become huge [10], which is mostly the case in a MEC environment. DRL is heavily used in recent proposals to address this issue. In [24], a DQN-based algorithm was introduced to obtain the optimal computation offloading policy. The objective was to minimize the long-term cost based on the channel characteristics between the mobile user and the base station, the energy queue and the task queue states. The same problem is solved in [3] using the DDQN algorithm, which gave significant improvement in computation offloading performance. The authors in [1] used the DQN algorithm to design a joint task offloading and bandwidth allocation decision in order to minimize the overall offloading cost in terms of energy,

computation and delay. The authors in [25] applied DRL to intelligent Internet of Things (IoT) inside a MEC with massive MIMO network. A DQN algorithm was proposed to learn offloading decisions in order to improve the system performance and reduce the latency and energy consumption. However, the cost function and the MDP formulation are different compared to our work. In fact, to minimize their cost function, the authors optimized the bandwidth allocation based on some deterministic predefined criteria before optimizing the offloading strategy. In addition, they did not consider the varying wireless channel conditions in their MDP formulation. Moreover their DQN neural network architecture has not been fully specified. To overcome these limitations, in our work we reformulate the problem without predefined criteria while taking into consideration the stochastic wireless channel under the MIMO technology. To use this work as a benchmark, we followed the authors' approach to apply the DQN algorithm with a clear presentation of our neural network architecture. Furthermore, we introduced a new PPO algorithm for the computation offloading in order to solve the limitation of the discrete action space in the DQN algorithm. The simulation results (see Section 5) show that our PPO algorithm has better performance.

3. System model and problem formulation

In a MEC architecture, multiple users are connected to a base station equipped with a high performance server (Fig. 1). The servers are deployed at the edge of the network to provide cloud-like services in close proximity of end-users. Thus, the users offload their computation-intensive tasks to the close edge server they are connected to, instead of remote cloud infrastructure. Thereby, enabling a wide range of applications and services to work in real or near-real time performance.

In our problem formulation, we consider a set of K mobile single-antenna users $\mathcal{U} = \{u_1, \dots, u_K\}$ communicate with a massive MIMO based MEC server through a Base Station (BS) equipped with N antennas.

The base station is equipped with a very large number of antennas to increase the spectral and energy efficiency. Tens, hundreds, or even thousands of antennas are commonly used in massive MIMO systems [38]. Usually, the number of antennas deployed are largely greater than the number of users $N \gg K$, which is the case considered in the literature, for example: [39–41]

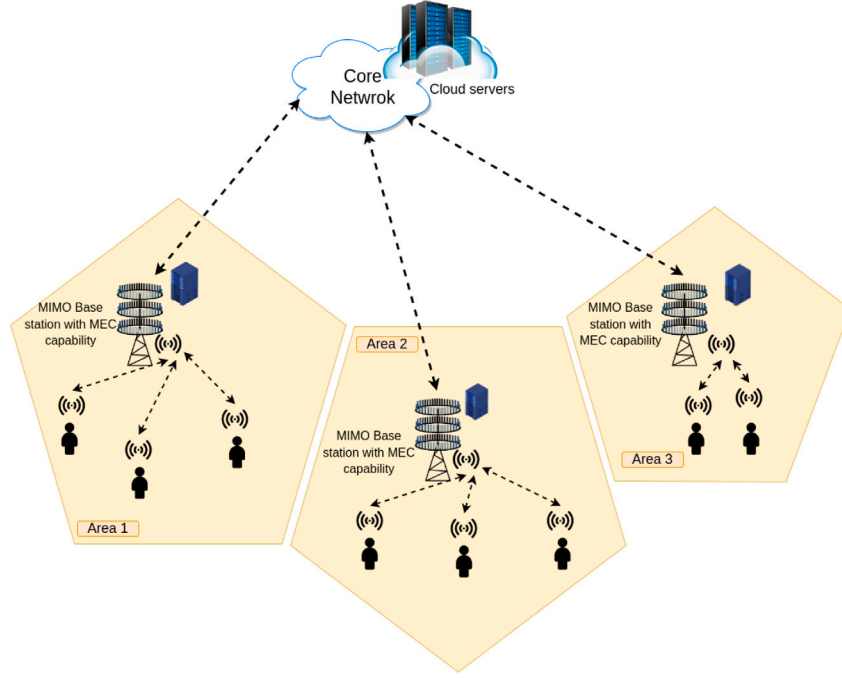


Fig. 2. Generalization aspect of the problem.

For simplicity but without lack of generality, we consider one BS (i.e., one area). Generalizing to multiple BSs is straightforward because they are independent and different base stations serve different users at each time moment. This generalization is with regard to the base stations per area, not the servers side, which is not studied in this paper. In contrast to a multi-server MEC environment where the users can offload to multiple servers, in our problem, we consider a massive MIMO setting where different base stations serve different users at each time moment. Fig. 2 shows an overview of the generalization aspect of the problem. In every area, the goal is to optimize the offloading operation at each user's MD which is connected to one base station at a time.

Based on previous work that studied the MEC offloading problem [42,43], we also adopt a discrete time model, so the system time T is split into equal time slots of length t_0 , $T = \{0, t_0, 2t_0, \dots\}$ and at each time slot, the MD should decide about the optimal strategy for the computational task. t_0 depends in fact on the type of application being executed and how the CPU is constructing the task queue. Concretely, in our paper, the communication between the users and the base station is a wireless channel (typically the Long Term Evolution (LTE) associated with the 4G and 5G wireless standards). Thus, as we do not have any constraint on the type of the computational task, the slot length t_0 is set in simulations to 1 ms, which is the typical Transmission Time Interval (TTE) in LTE wireless networks as argued in [44].

The variables used in our problem formulation are listed in Table 1.

3.1. Wireless channel model

In the rest of the paper, if a variable is parameterized by k , then it refers to the MD, otherwise it refers to the BS. At each time $t \in T$, a user $u_k \in \mathcal{U}$ transmits a signal $x_k(t) \in \mathbb{C}$ to the BS, where $\mathbb{C}$ is the set of complex numbers. Let $\mathbf{x}(t) = [x_1(t), x_2(t), \dots, x_K(t)]^T \in \mathbb{C}^{K \times 1}$ be the complex transmitted signal vector from all the users to the BS. Table 2 summarizes all the variables used in the wireless channel model.

The $(N \times 1)$ received signal vector in the uplink at the BS $\mathbf{y}(t) \in \mathbb{C}^{N \times 1}$ is expressed by:

$$\mathbf{y}(t) = \mathbf{H}(t)\mathbf{x}(t) + \mathbf{b}(t)$$

where $\mathbf{H}(t) \in \mathbb{C}^{N \times K}$ is the Channel State Information (CSI) matrix of complex channel gains between all the users and the BS:

$$\mathbf{H}(t) = \begin{bmatrix} h_{11}(t) & h_{12}(t) & \dots & h_{1K}(t) \\ h_{21}(t) & h_{22}(t) & \dots & h_{2K}(t) \\ \vdots & \vdots & \ddots & \vdots \\ h_{N1}(t) & h_{N2}(t) & \dots & h_{NK}(t) \end{bmatrix}$$

$h_{nk}(t) \in \mathbb{C}$ represents the channel response between the antenna of user k and the n th BS antenna, and $\mathbf{b}(t) = [b_1(t), b_2(t), \dots, b_N(t)]^T \in \mathbb{C}^{N \times 1}$ is the Additive White Gaussian Noise (AWGN) which follows a complex normal distribution with zero mean and variance σ^2 : $\mathbf{b}(t) \sim \mathcal{CN}(0, \sigma^2 \mathbf{I}_N)$ ($\mathbf{I}_N$ is the $N \times N$ identity matrix). We assume perfect CSI, and the Zero-Forcing (ZF) technique is adopted at the BS to suppress the inter-user interference. Hence, the ZF-detection matrix at the BS is given by the pseudo-inverse of its CSI Matrix $\mathbf{H}$ [32]:

$$\mathbf{V}(t) = \mathbf{H}(t)^H (\mathbf{H}(t)\mathbf{H}(t)^H)^{-1} \quad (1)$$

Let $\mathbf{h}_k(t) = [h_{1k}(t), h_{2k}(t), \dots, h_{Nk}(t)]^T \in \mathbb{C}^{N \times 1}$ be the CSI between the user u_k and the BS and $\mathbf{v}_k$ the k th row of $\mathbf{V}$, the normalized effective channel gain for user u_k can be defined as:

$$\Gamma_k(t) = \frac{|\mathbf{v}_k(t)\mathbf{h}_k(t)|^2}{\sigma^2 |\mathbf{v}_k(t)|^2} \quad (2)$$

Accordingly, the achievable channel capacity at user u_k is given by:

$$C_k(t) = W \log_2(1 + p_k^r(t)\Gamma_k(t)) \quad (3)$$

where W is the channel bandwidth in Hertz and $p_k^r(t) \in [0, P_k^r]$ is the transmit power of user u_k with P_k^r being its maximum transmission power constraint.

3.2. Computation model

At each time $t \in T$, each user $u_k \in \mathcal{U}$ has a computationally intensive task of size $m_k(t)$ bits to be executed. We assume that the tasks are coming from an application that can be partitioned, and therefore the user can offload part of it to the MEC server to assist with the computation. Some examples of such applications include image compression applications and virus detection software [20]. The MEC

Table 1
Problem formulation notations.

Symbol	Meaning
BS	Base station
N	Number of antennas at the BS
n	Antenna n at the base station, $1 \leq n \leq N$
$\mathcal{U}$	Set of all users connected to the BS
K	Number of users
u_k	User k in the network, $1 \leq k \leq K$
t_0	Length of a time slot
T	System time $\{0, t_0, 2t_0, \dots\}$
t	Time step $t \in T$

Table 2
Wireless channel model notations.

Symbol	Meaning
$\mathbb{C}$	Set of complex numbers
x_k	Transmitted complex signal of user u_k
x	Transmitted signal vector from all the users to the BS
y	Received signal in the uplink at the BS
H	Channel state information matrix between the users and the BS
h_{nk}	Channel response between user k and the n th BS antenna
b	Additive White Gaussian Noise (AWGN) vector
σ^2	Variance of the AWGN
$\mathcal{CN}$	Complex normal distribution
I_N	Identity matrix with N rows and N columns
H^H	Hermitian matrix of H
$()^T$	Matrix transpose
V	ZF-detection matrix
Γ_k	Normalized effective channel gain for user u_k
C	Achievable channel capacity at user u_k
W	Channel bandwidth
p_k^t	Transmission power of user u_k
p_k^{tr}	Maximum transmission of user u_k

servers are generally equipped with higher computational capability than the MDs and they can serve multiple users at the same time. Motivated by the work in [27], we assume that the MD of each user is equipped with the dynamic voltage and frequency scaling (DVFS) technology, where the CPU frequency can be adjusted depending on the computation needs. Table 3 contains all the variables used in our computation model.

Let $f_k(t) \in [0, F_k]$ be the CPU frequency allocated to execute a task at time t with F_k being the maximum frequency constraint of the MD. We denote by $m_k^{lo}(t)$ the amount of data bits that the user can execute locally, hence:

$$m_k^{lo}(t) = \frac{t_0 f_k(t)}{\omega_k} \quad (4)$$

where ω_k (cycles/bit) is the number of cycles required to execute one bit of data.

We model the power consumption of the CPU executing this amount of data as in [27,45] by:

$$p_k^{lo}(t) = \eta f_k(t)^3 \quad (5)$$

where η is a coefficient that depends on the hardware architecture of the MD.

Let $m_k^{tr}(t)$ be the amount of data bits of the offloaded part to the edge server. Assuming that the latency resulting from the MEC computation and the downlink transmission is negligible, the time delay to get the result from the edge server is given by:

$$d_k(t) = \frac{m_k^{tr}(t)}{C_k(t)} \quad (6)$$

This assumption has been widely utilized in the MEC literature [46–48]. This is mainly due to two reasons:

- Because of the superior computation capabilities of the servers and the small sizes of the computation results, the delay caused

Table 3
Computation model notations.

Symbol	Meaning	Units
m_k	Size of a task of user u_k	bits
f_k	Frequency of the CPU of user u_k	Hz
F_k	Maximum frequency of the CPU of user u_k	Hz
m_k^{lo}	Size of the part of the task of user u_k that is computed locally	bits
ω_k	Number of CPU cycles required to execute one bit of data at the MD of user u_k	cycles/bit
p_k^{lo}	Local power consumption of user u_k	W
η	CPU hardware architecture coefficient	W s ³
m_k^{tr}	Size of the part of user u_k 's task that is offloaded to the edge server	bits
d_k	Time delay for the user u_k to get the computation results	s

by the MEC server to compute an offloaded task and the time for a user to download the computation results from a server are negligible.

- Because MEC servers are not constrained in terms of energy, the energy consumed by a MEC server to compute the offloaded activities, as well as the transmission energy consumption may be excluded.

3.3. System cost

To take into consideration both the energy consumption and the offloading latency, we modeled the problem cost as a weighed sum of the total power consumption and the time delay resulting from the offloading operation. As a result, we define the system cost as:

$$\Phi_k(t) = \alpha(p_k^{lo}(t) + p_k^{tr}(t)) + (1 - \alpha)d_k(t) \quad (7)$$

where α is a positive number between 0 and 1 to determine the weight of the trade-off between two factors: the energy and the delay. It is a hyper-parameter which we can tune to achieve a balance between these two factors. In our simulations (see Sections 5.1 and 5.2), we will consider different scenarios of α and we will investigate its effect on the offloading strategy.

Our objective is to minimize the long-term cost by dynamically allocating the CPU frequency $f_k(t)$ for local computation and the transmit power $p_k^{tr}(t)$ for the offloading computation. Considering the maximum frequency and power constraints, the corresponding problem can be formulated as:

$$\arg \min_{f_k(t), p_k^{tr}(t)} \Phi_k(t), \forall t \in T$$

s.t

$$f_k(t) \in [0, F_k] \quad \forall t \in T \quad (8)$$

$$p_k^{tr}(t) \in [0, p_k^{tr}] \quad \forall t \in T \quad (9)$$

4. DRL-based strategies for cost minimization

DRL is an active research field of machine learning that combines the RL framework with artificial neural networks to optimize an objective where the dynamics of the environment keeps changing [49–51]. In this section, we present briefly the RL framework and introduce our two strategies to solve the optimization objective in our problem formulation.

4.1. Background

DRL is based on the standard RL setup which consists of an agent interacting with its environment and, upon observing the results of its actions through a received reward from the environment, can learn the optimized behavior. The RL problem is formalized mathematically as a MDP which is a discrete time stochastic control process defined as a 5-tuple $(S, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$. S is a set that represents all the possible states of the environment. $\mathcal{A}$ is the action space, i.e., the set of actions that the agent can take. $\mathcal{T} : S \times \mathcal{A} \times S \rightarrow [0, 1]$ is the transition probability matrix that maps each state-action pair at time t to a probability distribution over states at time $t + 1$. In most real-life applications, $\mathcal{T}$ is not known due to the fact that it is hard to come up with a model of the environment, which is the case also in our stochastic offloading problem. This is where model-free RL algorithms [52] can be used. $\mathcal{R} : S \times \mathcal{A} \times S \rightarrow \mathbb{R}$ is a reward function that outputs a scalar value $r(t)$ based on the current state at time t , $s(t)$, the action taken $a(t)$ and the next state $s(t + 1)$. $\gamma \in [0, 1]$ is a discount factor which reflects how much the agent cares about future rewards depending on how γ is close to 1. In general, the agent chooses its actions based on a policy π which is a mapping from the state space to a probability distribution over actions. Following a policy π , the sequence of states, actions and rewards resulting from the interaction of the agent with the environment constitutes a trajectory (or episode) of that policy. The discounted cumulative reward resulting from the trajectory is called a return $G = \sum_{t=0}^{+\infty} \gamma^t r(t + 1)$. The end goal of RL algorithms is to find an optimal policy π^* that maximizes the expected return [53]. In order to apply DRL on our problem, we need to formalize it as a MDP first. In the sequel, we provide the details of our formalism.

4.2. MDP formulation

We model and formalize our problem as a MDP, taking into consideration the stochastic wireless channel conditions and the variation of the tasks to be executed. In our formulation, we consider the problem from the user's perspective, where the agent (the user's mobile device) will try to find the optimum policy to minimize its cost function. The fundamental blocks of our MDP are defined as follows:

4.2.1. State space

The state space is a representation of the environment the agent tries to interact with. In our problem, the agent needs to observe, at each time slot t , the actual task $m_k(t)$ that needs to be executed, and the wireless channel state information $H_k(t)$ in order to optimally decide the offloading part to the MEC server. As a result, each user $u_k \in \mathcal{U}$ will have its own internal representation of the environment. We define $s_k(t)$ the representation of the environment from the u_k user's perspective at each timestamp $t \in T$ by the vector: $s_k(t) = (m_k(t), h_{1k}(t), h_{2k}(t), \dots, h_{Nk}(t))$ where $h_{ik}(t), i \in \{1, 2, \dots, N\}$ is the CSI between the user u_k and the i th BS antenna.

4.2.2. Action space

In our MEC system model, the agent needs to find the optimal offloading strategy by allocating the optimal local execution power and the optimal offloading power for remote MEC execution. Therefore, for a user u_k , the action at time t is defined by the tuple $a_k(t) = (f_k(t), p_k^{lr}(t))$ where $f_k(t) \in [0, F_k]$ is the allocated CPU frequency for the local execution and $p_k^{lr}(t) \in [0, P_k^{lr}]$ is the transmission power for the offloading execution. However, this action space is defined on the continuous domain and some RL algorithms such as DQN do not support continuous action space. As a workaround to this limitation, we adopt the same approach as in [54] and we devise the above continuous domains into $l \in \{1, 2, \dots\}$ levels. Thus, we define the set of available CPU frequencies as $\mathcal{F} = \{0, \frac{F_k}{l}, \frac{2F_k}{l}, \dots, F_k\}$ and the set of available transmission powers as $\mathcal{P} = \{0, \frac{P_k^{lr}}{l}, \frac{2P_k^{lr}}{l}, \dots, P_k^{lr}\}$. Accordingly, we define the l -level action space in the discrete case as the Cartesian product of the two sets $\mathcal{F}$ and $\mathcal{P}$ as follows: $\{(f_k(t), p_k^{lr}(t)) | f_k(t) \in \mathcal{F} \text{ and } p_k^{lr}(t) \in \mathcal{P}; \forall t \in T\}$

4.2.3. Reward function

Based on the state representation and the chosen action, the agent receives a reward that guides the learning process. The agent goal is to choose the action that will give the highest reward. The reward function is generally associated with the objective function. In our formulation, the goal is to achieve the minimum overall cost of user u_k . Accordingly, the value of the reward must be negatively correlated to the value of the cost. We define the immediate reward of a user $u_k \in \mathcal{U}$ at time slot $t \in T$ as $r_k(t) = -\Phi_k(t)$, where $\Phi_k(t)$ is the total cost defined in Eq. (7).

4.3. DQN strategy

Q-learning is one of most popular RL algorithms [53]. It is based on the action-value function (or the Q-function) $Q^\pi(s, a)$ which measures the expected return from state s and taking action a following the policy π . The optimal action-value function $Q^*(s, a)$ which gives the maximum expected return over all policies can be obtained using the following Bellman Optimality Equation:

$$Q^*(s(t), a(t)) = \mathbb{E}[r(t) + \gamma \max_{a(t+1)} Q^*(s(t+1), a(t+1))] \quad (10)$$

The basic idea behind Q-learning is to use Eq. (10) as a simple iterative update used in the tabular Q-Learning algorithm to update the Q-values in an iterative way in order to converge towards the optimal policy as shown by Eq. (11):

$$Q(s(t), a(t)) \leftarrow Q(s(t), a(t)) + \beta[r(t) + \gamma \max_{a(t+1)} Q(s(t+1), a(t+1)) - Q(s(t), a(t))] \quad (11)$$

where $\beta \in (0, 1]$ is the learning rate. Storing all Q-values in a table structure for each state-action pair, and using Eq. (11) plus an exploration and exploitation trade-off to ensure all the action space is explored, the Q-learning algorithm will converge eventually to the optimal Q-function [55].

It is worth noting that the state and action spaces grow with the complexity of the problem. Thus, using a table in the memory to store each state-action pair is very expensive in terms of computation, especially when trying to update the Q-values for each cell on the table. As a result, applying standard Q-learning in this case is time and memory consuming, and may even diverge [56]. This is known as the curse of high dimensionality problem [24]. In fact, the state space of our problem grows with the number of antennas at the base station. The action space increases as well depending on the l -level action we need (see the MDP design, Section 4.2).

To overcome this limitation, function approximation can be used to learn the Q-values. Recent advances in Deep Learning have revolutionized the field of RL, leading to the creation of the excited field of DRL. The usage of Deep Neural Network (DNN) as function approximator gives the agent the ability to learn from complex environments [57]. DQN [58] is one of the effective algorithms that combines DL with RL. As shown in Fig. 3, our DQN strategy uses DNN with weights θ to approximate the Q function. For every iteration, these weights get updated using gradient decent to converge towards the optimal Q values. Algorithm 1 provides a pseudo-code of our cost minimization strategy based on the DQN algorithm.

In Algorithm 1, we start by building a DNN used as a function approximator for the action-value function Q. The input of the model is a state vector, and the output is a layer containing as many neurons as the number of elements of our action space, which depends on the l -level action parameter l given as input (line 2). We then initialize a queue Δ as a reply buffer of size C given as input. It will be used to store the past experience of our agent, namely the state, action, reward and the next state observed by executing that action (line 3).

Inside the learning loop (line 4), which will be running for a maximum of I iterations (episodes) given as input, we start by resetting our environment to a random state s (line 5). Each episode is limited

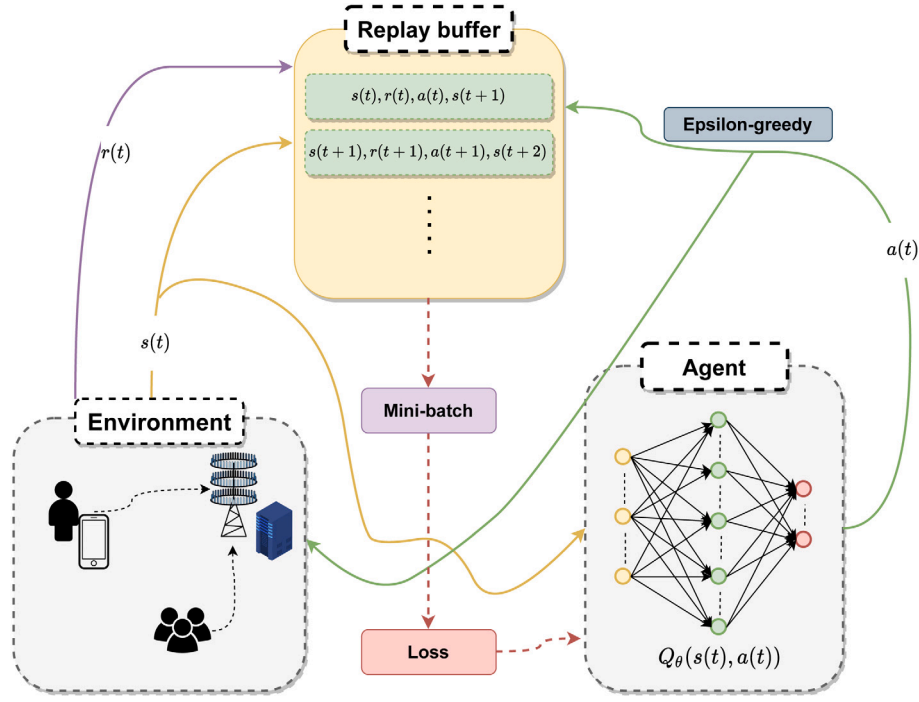


Fig. 3. The structure of our Deep Q-Network (DQN) strategy.

by L learning steps. In every step, we start by deciding on the action to take following the ϵ -greedy policy, which is an efficient method to achieve a trade-off between exploration and exploitation. Here ϵ is a number between zero and one. at the beginning, ϵ will be close to ϵ_0 , which we set to one ($\epsilon_0 = 1$) to allow more exploration. With every iteration, ϵ will decay according to a decay factor $\epsilon_d = 5000$ until it reaches $\epsilon_f = 0.001$ as the agent should use more the knowledge learned through its experience with the environment. We use the exponential decay formula to decrease ϵ in every iteration, which is defined as $ExpD(\epsilon_0, \epsilon_f, \epsilon_d) = \epsilon_f + (\epsilon_0 - \epsilon_f) * \exp(-\frac{i+L*(iteration-1)}{\epsilon_d})$ (line 8). Then, we generate a random number ξ between zero and one. If $\xi \leq \epsilon$, the algorithm picks a random action a from the action space (lines 10), and in this case, we say that the agent is exploring. Otherwise, the algorithm picks the action with the maximum Q-value from the neural network (lines 12), and in this case we say that the agent is exploiting, i.e. using the results of the training process so far.

Executing the selected action on the environment, the agent gets the reward r and the next state s' . This transition of (state, action, reward, next state) will be pushed to the replay buffer Δ (lines 14–16). If Q has enough values, we sample a random mini-batch of size B from it, in the hope of making our data more independent and identically distributed (i.i.d) to provide better convergence performance when training the neural network (line 18). The replay buffer Δ will play the role of a data-set as if we are in a supervised learning context, we calculate the predicted Q-values using the states s stored in Δ (line 19), then we forward the next states s' through the network to get the Q-values of the next states (line 20). The rewards in line 21 are extracted from the mini-batch Δ' and stored in a list R . The rewards in the mini-batch are calculated in line 15 based on Section 4.2.3. We can then perform the element wise operations to calculate the expected Q-values using the Bellman equation (lines 22). In our implementation, we used the mean squared error to calculate the loss between the expected Q-values and the predicted Q-values as the following: $MSE(Y, \hat{Y}) = \frac{1}{|Y|} \sum_{i=1}^{|Y|} (Y_i - \hat{Y}_i)^2$ (line 23) and we used the Adam optimizer for the gradient decent algorithm to backpropagate this loss through the neural network (lines 24). At the end of the iteration, we remove the old element from the replay buffer Δ to free the space to the next transition, keeping the agent on softly updating and learning as new data are being pushed to

the buffer (line 25), and we assign the next state value to the actual state variable (line 26).

4.4. PPO strategy

DQN has solved the problem of learning the Q-values for complex environments with high dimensional state space using Deep Neural Networks. However, it can only handle discrete and low-dimensional action spaces. Recently, Schulman et al. proposed the PPO algorithm [59], a cutting-edge DRL method which can be applied to discrete as well as continuous state and action spaces [60]. Moreover, the authors argue that it outperforms the other algorithms on their benchmark while providing a good balance between ease of tuning, efficient sampling and simple implementation.

PPO is based on the Actor–Critic approach. As shown in Fig. 4, the Actor–Critic framework uses two separate neural networks: the Actor and the Critic. The former represents directly the policy π of the agent which controls the offloading scheme and decides on the amount of data that should be computed locally and the transmission power of the offloaded part to the MEC server. It receives the state containing the current task size and the wireless channel vector as input, and outputs a probability distribution over the actions. The agent chooses its next action by sampling from this distribution. The latter is used to approximate the value function $V^\pi(s) = \mathbb{E}_\pi \{ \sum_{k=0}^{\infty} \gamma^k r(t+k+1) | s(t) = s \}$, which estimates how rewarding a given state can be for the agent. It receives the same state of the Actor as input and outputs the estimated value of $V^\pi(s)$. In other words, Actor–Critic is a combination of policy optimization and value optimization, the Actor decides which action to take and the Critic evaluates this action and tells the Actor how it should adjust.

During the learning process, PPO stores the interaction of the agent with the environment into a memory. This memory will be used to update the Actor–Critic networks. Unlike DQN's experience replay buffer, the PPO algorithm uses all the memory and not just a sample mini-batch to update the neural networks. Moreover, after each update, the entire memory is cleared and not just the oldest element. The updated networks are then used to collect new experience and refill the memory.

Algorithm 1 DQN strategy for cost minimization

```

1: Input: reply buffer capacity  $C$ , discount factor  $\gamma$ , maximum number
   of iterations  $I$ , episode length  $L$ , mini-batch size  $B$ ,  $l$ -level action  $l$ 
2: Create a Deep Neural Network and initialize its weights  $\theta$  with
   random values  $Q(\theta)$ 
3: Initialize a queue  $\Delta$  as a reply buffer with capacity  $C$ ;
4: for iteration  $\leftarrow 1 \dots I$  do
5:    $s \leftarrow$  Get the state vector as defined in our MDP
6:   for  $i \leftarrow 1 \dots L$  do
7:     # Following  $\epsilon$ -greedy policy
8:      $\epsilon \leftarrow \text{ExpD}(\epsilon_0, \epsilon_f, \epsilon_d)$ 
9:     if Random number  $\xi \leq \epsilon$  then
10:       $a \leftarrow$  Select randomly a frequency and a power
        transmission tuple from the discrete action
        space as defined in the MDP
11:     else
12:       $a \leftarrow \arg \max Q(s; \theta)$ 
        # Get a predicted action by forwarding the state  $s$  through
        the neural network
13:     end if
14:     Execute the action  $a$  on the environment
15:     Get the reward  $r$  and the next state  $s'$ 
16:     Push  $(s, a, r, s')$  into  $\Delta$ 
17:     if  $\text{length}(\Delta) \geq B$  then
18:        $\Delta' \leftarrow$  Sample a random mini-batch of size  $B$ 
        from  $\Delta$ 
19:        $Y \leftarrow$  Get the Q-values by forwarding the states
         $s \in \Delta'$  through the neural network
20:        $X \leftarrow$  Get the next Q-values by forwarding next
        states  $s' \in \Delta'$  through the neural network
21:        $R \leftarrow$  All the rewards  $r$  from our mini-batch  $\Delta'$ 
22:        $\hat{Y} \leftarrow R + \gamma X$  # Element wise operations
23:        $\text{loss} \leftarrow \text{MSE}(Y, \hat{Y})$ 
24:       Backpropagate the loss using gradient decent
        with the Adam optimizer
25:       Dequeue the old element from  $\Delta$ 
26:        $s \leftarrow s'$ 
27:     end if
28:   end for
29: end for

```

Policy optimization is based on the policy gradient methods, which use a stochastic gradient ascent algorithm to maximize an objective function. The most common objective function has the following form [59]:

$$L^{PG} = \mathbb{E}_t[\log \pi_\theta(a(t)|s(t))\hat{A}(t)] \quad (12)$$

and the gradient estimator is given by :

$$g = \mathbb{E}_t[\nabla_\theta \log \pi_\theta(a(t)|s(t))\hat{A}(t)] \quad (13)$$

where π_θ is the policy neural network with parameters denoted by θ and $\hat{A}(t)$ is an estimator of the advantage function defined by:

$$A(t) = Q(s(t), a(t)) - V(s(t)) \quad (14)$$

However, this approach can be unstable due to the large updates when performing multiple optimization steps on the policy [59]. To solve this issue, PPO provides another objective function in order to limit the new policies to get far from the old policies. Let $\rho_t(\theta)$ denote the probability ratio between the new policy π_θ and the old policy $\pi_{\theta_{old}}$:

$$\rho_t(\theta) = \frac{\pi_\theta(a(t)|s(t))}{\pi_{\theta_{old}}(a(t)|s(t))} \quad (15)$$

The new objective function is given by:

$$L = \mathbb{E}_t[\min(\rho_t(\theta)\hat{A}(t), \text{clip}(\rho_t(\theta), 1 - \epsilon_c, 1 + \epsilon_c)\hat{A}(t))] \quad (16)$$

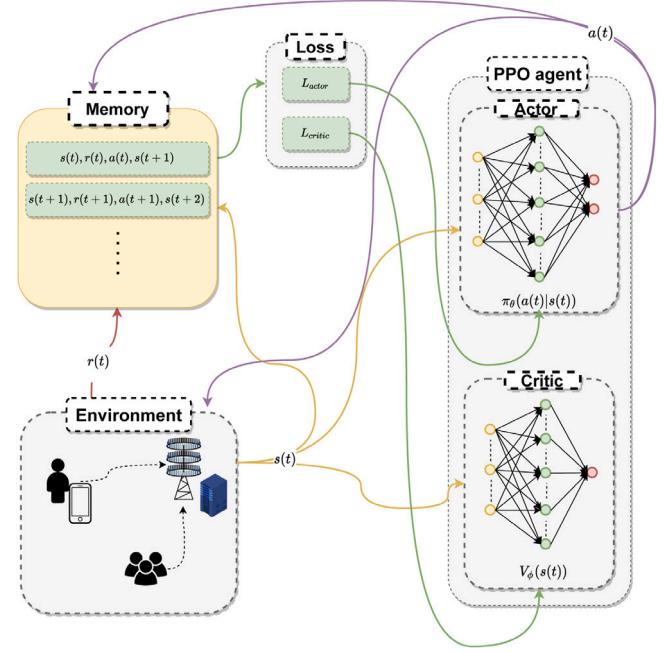


Fig. 4. The structure of our Actor-Critic PPO strategy.

where ϵ_c is a hyperparameter between zero and one, and clip is a function that clips the probability ratio to keep it inside the interval $[1 - \epsilon_c, 1 + \epsilon_c]$. The intuition behind this objective function is to keep the old and new policies close to each other by taking the minimum between the unclipped term and the clipped term inside some small interval controlled by ϵ_c . As far as the advantage function, there are several function estimators [61], PPO uses a truncated version of the Generalized Advantage Estimator (GAE):

$$\hat{A}(t) = \delta(t) + (\gamma\lambda)\delta(t+1) + \dots + (\gamma\lambda)^{T-t-1}\delta(T-1), \quad (17)$$

$$\text{where } \delta(t) = r(t) + \gamma V(s(t+1)) - V(s(t)) \quad (18)$$

with T is a given length and λ is a hyperparameter.

Our PPO-based strategy for cost minimization is illustrated in Algorithm 2. We start by creating two randomly initialized DNNs: one for the Actor (we denote its parameters by θ) and the other for the Critic (we denote its parameters by ϕ) (lines 2–3). The input of the Actor and the Critic networks is the state vector as defined in our MDP (Section 4.2). The output of the Actor is a layer containing two neurons according to our action tuple. On the other hand, the output of the Critic is a layer that contains one neuron representing the estimation of the value function of the state given as input. In line 4 we create a memory list to store the experience of the agent. This will be used later to fit the Actor and Critic networks. We run the algorithm for E epochs given as input (line 5). Inside this learning loop, we start by gathering the experience by running the policy π_θ in the environment for S time-steps (line 6). First, the agent observes the actual state of the environment, which consists of the task that needs to be executed and the channel state information between the user and the edge server (line 7). Then, an action is sampled from the probability distribution based on the predictions of the policy network π_θ as in line 8. It consists of the CPU frequency allocated for the task and the transmission power of the offloaded part. This action is used then to compute the size of the task that can be executed locally based on the CPU frequency allocated, and the delay generated from the offloading operation according to the transmission energy (lines 9–10). In line 11, the system cost $\Phi(t)$ is calculated using Eq. (7) and the aforementioned computed values, and the negative of this value is assigned to the reward as in line 12. We finish the iteration by storing the collected data into the memory. After

collecting the data, the algorithm uses the entire memory to update the DNNs. First, it uses the rewards stored in the memory to build a list of discounted cumulative rewards $\hat{R}$ to be used as a target to fit the Critic network (line 15). Then, the advantage function is estimated using GAE for each time-step along the memory, based on the predictions of the Critic network V_ϕ (line 16). Here the estimate advantage list $\hat{A}$ will be used to compute the Actor loss using Eq. (16) as in line 13. Next, we forward the states from the memory through the value function network V_ϕ to create the list R . This is used to find the loss of the Critic network by computing the Mean-Squared Error (MSE) between R and $\hat{R}$:

$$MSE(R, \hat{R}) = \frac{1}{|R|} \sum_{i=1}^{|R|} (R_i - \hat{R}_i)^2 \quad (19)$$

where $|R|$ is the number of elements in the list R . Finally, the algorithm updates the DNNs of the Actor and the Critic. We use the gradient ascent algorithm with the Adam optimizer to update the Actor network π_θ since this is a maximization problem towards the parameter θ that produces the highest return (line 19). Then, we use gradient descent with the Adam optimizer to fit the value function represented by the Critic network (line 20). We repeat this process for a number of epochs until convergence.

Algorithm 2 PPO strategy for cost minimization

```

1: Input: Number of epochs  $E$ , Steps per epoch  $S$ , clip hyperparameter  $\epsilon_c$ , GAE factor  $\lambda$ 
2: Create the Actor Network (the policy  $\pi$ ) and initialize its weights  $\theta$  with random values
3: Create the Critic Network (the value function  $V$ ) and initialize its weights  $\phi$  with random values
4: Create the memory list
5: for  $e \leftarrow 1 \dots E$  do
6:   for  $t \leftarrow 1 \dots S$  do
7:      $s \leftarrow$  Get the state vector as defined in Section 4.2
8:      $a \leftarrow$  Sample the allocated frequency and transmission power from the probability distribution predicted by the Actor network  $\pi_\theta$ 
9:     Compute the size of the part allocated for local execution using Eq. (4)
10:    Compute the delay of the offloaded part using Eq. (6)
11:    Calculate the system cost  $\Phi$  using Eq. (7)
12:     $r \leftarrow -\Phi$ 
13:    Store the experience in the memory
14:  end for
15:  # Using the experience stored in the memory we compute the following:
16:   $\hat{R} \leftarrow$  Compute the discounted cumulative rewards
17:   $\hat{A} \leftarrow$  Estimate the advantage function using (GAE) (Eq. (17)) with the help of the critic network  $V_\phi$ 
18:   $L_{actor} \leftarrow$  Compute the Actor loss using Eq. (16)
19:   $L_{critic} \leftarrow MSE(R, \hat{R})$ 
20:  #  $R$  obtained by forwarding the states through  $V_\phi$ 
21:  Update the Actor network  $\pi_\theta$  by backpropagating  $L_{actor}$  using stochastic gradient ascent with the Adam optimizer
22:  Update the Critic network  $V_\phi$  by backpropagating  $L_{critic}$  using gradient descent with the Adam optimizer
23: end for

```

5. Experiments and simulation results

In this section, we conduct simulation experiments to evaluate and compare the performance of our proposed solutions: DQN and PPO-based algorithms. We used Python as our programming language with

Table 4

Simulation parameters.

Parameter	Value
N	64
W	1 MHz
σ	10^{-9}
f_{max}	1.5 GHz
p_{max}^r	2 W
ω	500 cycles/bit
η	10^{-26}
t_0	1 ms

Pytorch, an open source library for machine and deep learning, to implement our solutions. The setup is as follows: we run our experiments on the Cedar Compute Canada cluster,¹ which is a High Performance Computing (HPC) resource having a total of 94,528 CPU cores, 1352 GPU devices and from 125 GB to 3022 GB of RAM and a total of 22926TB of storage capacity. The simulations are conducted with the help of the MIMO LAB Dataset [62], an open dataset collected on a 64-antenna base station. It consists of the CSI measurements of a Massive MIMO system and contains 252004 samples. Each sample represents one channel measurement resulting in a complex channel matrix of the form $H \in \mathbb{C}^{64 \times 100}$.

In our simulation, we set the number of antennas to $N = 64$, the channel bandwidth to $W = 1$ MHz and the variance of the AWGN to $\sigma^2 = 10^{-9}$ W. In addition, we assume that the MD is an average mobile phone, so we set the maximum CPU frequency to $f_{max} = 1.5$ GHz and the maximum transmission power to $p_{max}^r = 2$ W, the number of cycles required to execute one bit of data to $\omega = 500$ cycles/bit and the coefficient of the CPU power consumption to $\eta = 10^{-26}$ [27]. Moreover, we assume that the sizes of the input tasks are uniformly distributed between $m_{min} = 10$ Mbits and $m_{max} = 30$ Mbits as the work in [1]. Finally, the width of the time slots is set to $t_0 = 1$ ms. All the parameters are listed in Table 4. We compared our solutions with the following benchmarks: DDQN, GREEDY and RANDOM.

5.1. DQN simulation results

To implement DQN, we used a fully connected feed-forward Neural Network with 1 hidden layer having 64 neurons and ReLU (Rectified Linear Unit $ReLU(x) = \max(0, x)$) as the activation function. We set $\gamma = 0.99$ and the learning rate for the Adam optimizer to 10^{-3} . The replay buffer capacity is set to $C = 1000$ and the mini-batch size to $B = 900$.

Fig. 5 shows the convergence time of the DQN strategy for multiple levels of action. The results show that the size of the action space has an effect on the convergence time. The more the action space becomes larger, the more the algorithm takes time to train. In addition, when the level increases, the cost decreases. This is because the more we sample from the continuous interval of the action, the more the action space becomes larger, and more the algorithm has the chance to find a better action to minimize the cost.

We set the action level to 5 and we investigate the effect of the weight factor α used in Eq. (7) on the DQN strategy. In Fig. 6, we show that DQN converges for the different values of α . In addition, the weight factor α has an effect on the convergence time, the more we give more weight to the energy, the less the algorithm takes time to converge. We also noticed that when we give more weight to the delay ($\alpha = 0.4$), the system cost decreases but the algorithm takes more time to converge. On the other hand, if we give more weight to the energy, the system cost increases and the algorithm converges faster.

¹ Cedar - CC Doc, <https://docs.computecanada.ca/wiki/Cedar>, (Accessed on 07/05/2022).

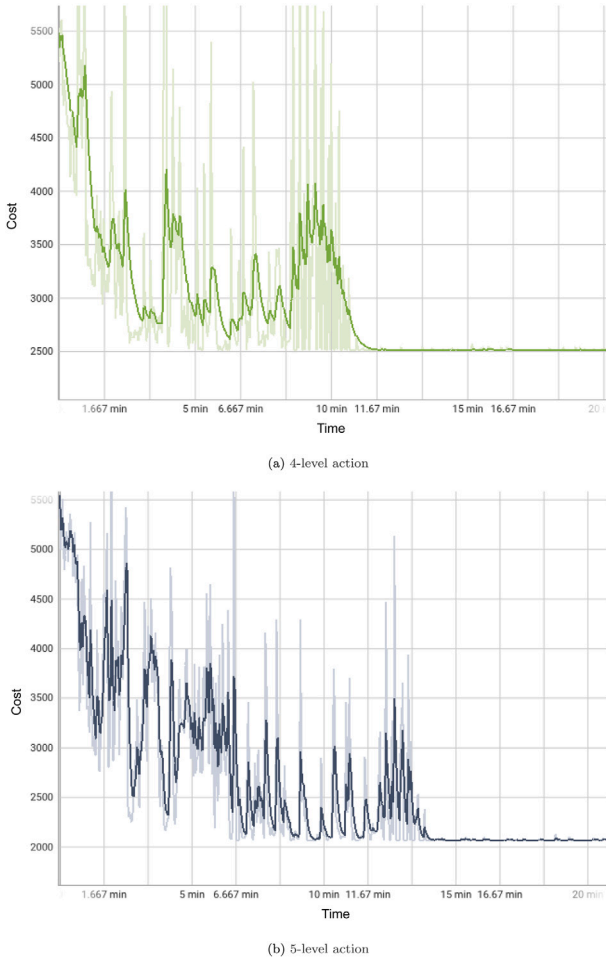


Fig. 5. Convergence time of the DQN strategy under different action levels.

Fig. 7 shows the effect of the mini-batch size B on the performance of the DQN strategy. The mini-batch is sampled randomly from the replay buffer to train the neural network. From the results, we can see that changing the mini-batch size has different effects on the convergence performance. For instance, when the mini-batch size is 900, the algorithm converges gradually towards a minimum cost and then it shows a stable performance. When the mini-batch size is 950, the algorithm does not even converge and it seems to have a random behavior. Meanwhile, when the batch-size is 1000, the algorithm seems to converge again but quickly and towards another different cost with huge margin difference with the 900 performance.

From the previous results, we notice that the main advantage of the DQN algorithm is that it converges quickly. On the other hand, besides the limitation of the sampling mechanism, the DQN algorithm is very sensitive to the hyper-parameters tuning, changing one parameter can lead to a different behavior and therefore a different performance.

5.2. PPO simulation results

In the implementation of our PPO-based solution, we used two fully connected feed-forward neural networks for the Actor and the Critic respectively. Each neural network has two hidden layers consisting of 64 neurons each, and having tanh (the hyperbolic tangent function) as the activation function. In addition, we set $\gamma = 0.99$, the GAE hyperparameter $\lambda = 0.97$, and the clip ratio $\epsilon_c = 0.2$. For the learning rates, we set the Actor learning rate to 3×10^{-3} and the Critic learning rate to 10^{-4} . Finally, we run the algorithm for $E = 250$ epochs with $T = 1000$ steps per epoch.

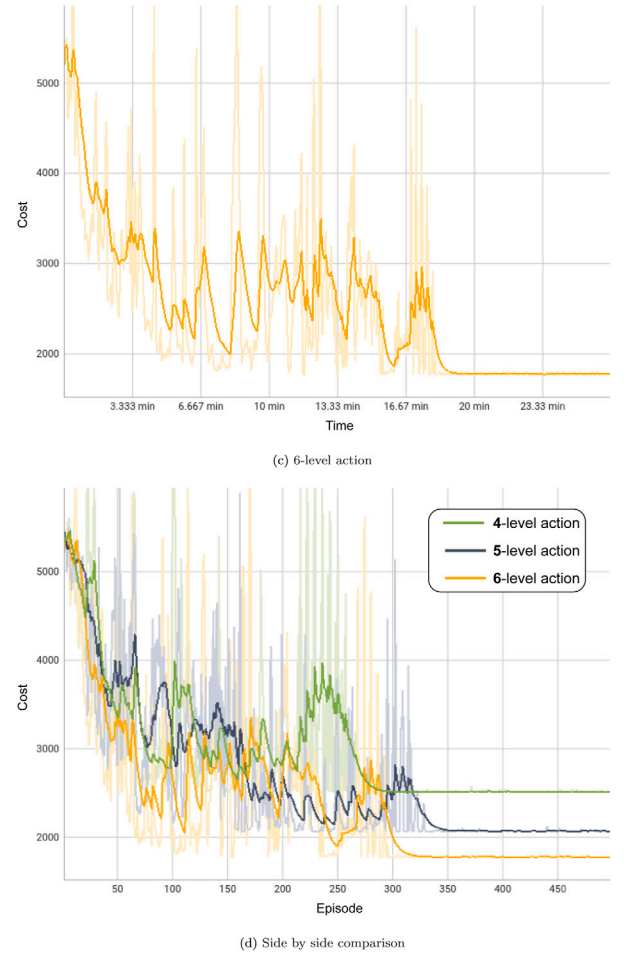


Fig. 5. (continued).

First, we study the convergence of our PPO strategy under $\alpha = 0.5$. As depicted in Fig. 8, the algorithm reaches the minimum after one hour of training and it continues improving with a slow rate after that.

In Fig. 9, we investigate the effect of the weight factor α on the performance of the PPO algorithm. The results show a noticeable stable performance for different values of α and each takes almost the same time to converge. In addition, the algorithm converges towards the same minimum cost in contrast with the DQN strategy, which means that even with more weight on the energy, the algorithm found an action in the continuous action space that could minimize the cost more. Unlike the DQN strategy which has a limited number of actions to choose from.

Fig. 10 shows the performance of our PPO algorithm for different values of the parameter S (steps per epoch) which also represents the length of the PPO memory. For every epoch, the training memory will contain S steps to train the Actor and the Critic networks. The results show that there is no significant effect on the convergence time of the PPO algorithm when slightly changing S in contrast with the DQN algorithm. Moreover, PPO converges towards the same minimum under the three different values of S , which means that it is more stable and not very sensitive to hyper-parameters tuning.

5.3. Performance comparison

In order to examine the performance of the proposed solutions, we compare them with different schemes by calculating the system cost resulting from applying each of the following strategies:

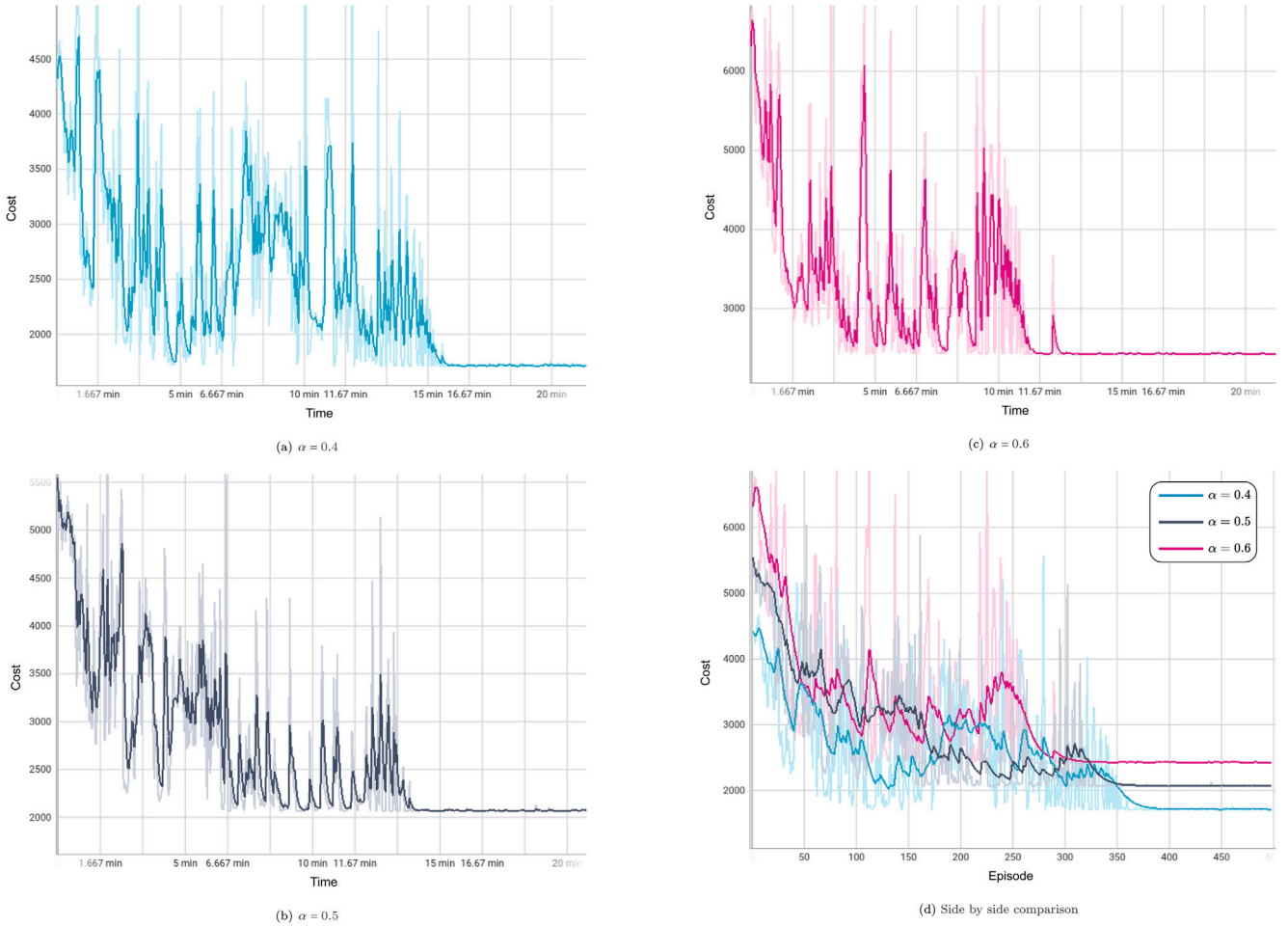


Fig. 6. Convergence time of the DQN strategy under different values of α .

- The PPO strategy: the trained model of our PPO algorithm.
- The DQN strategy: the trained model of the DQN algorithm.
- The DDQN strategy: proposed in [3], which is an adaptation of the DQN algorithm to solve the problem of overestimation. It uses two separate neural networks to represent the action-value function and each of which is used to update the parameters of the other [63].
- The GREEDY strategy: chooses the action with the highest performance.
- The RANDOM strategy: selects random action for each task size.

Fig. 11 shows a side-by-side comparison of the PPO, DQN and DDQN strategies with regards to the training performance. the PPO strategy outperforms both benchmark solutions in terms of the minimum cost. Furthermore, the DDQN strategy does not show any improvement over the DQN strategy as they converge towards the same minimum cost. On the other hand, the PPO algorithm takes way more time to converge compared to the two benchmarks, which is reasonable because of its Actor-Critic architecture. Also, the double neural network architecture introduced in the DDQN algorithm makes the training time taking longer compared to the DQN algorithm.

Fig. 12 shows the comparison results between the five strategies. We vary the task size m (Mbits) inside our interval of simulation and we measure the system cost resulting from each scheme. The results show that our proposed solutions give the best performance among the other strategies. They also show that the PPO strategy gives the best result compared to the benchmark DQN and DDQN strategies.

Fig. 6. (continued).

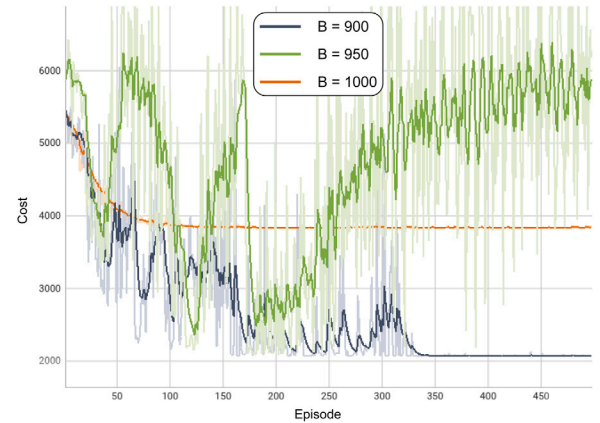


Fig. 7. DQN Sensitivity to the mini-batch hyper-parameter tuning.

6. Conclusion and future work

In this paper, we addressed the computation offloading problem in a mutative MIMO-based MEC environment considering the stochastic time-varying wireless channels and computation task arrivals. We formulated our problem with the objective of minimizing the long-term cost in terms of energy consumption and offloading delay under the constraints of the limited computation capacity and transmission energy of the MDs. A MDP has been designed for the problem, and two DRL-based strategies have been introduced to learn a dynamic

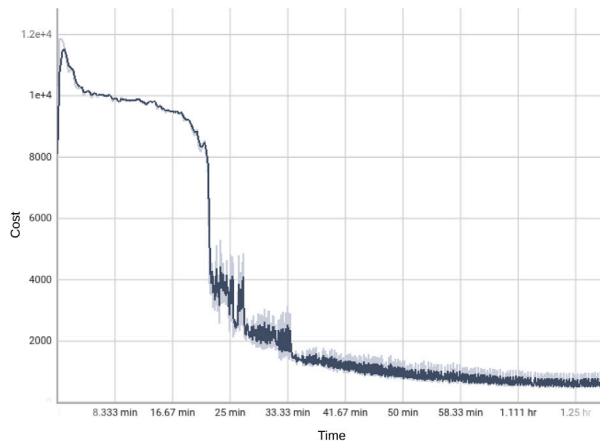
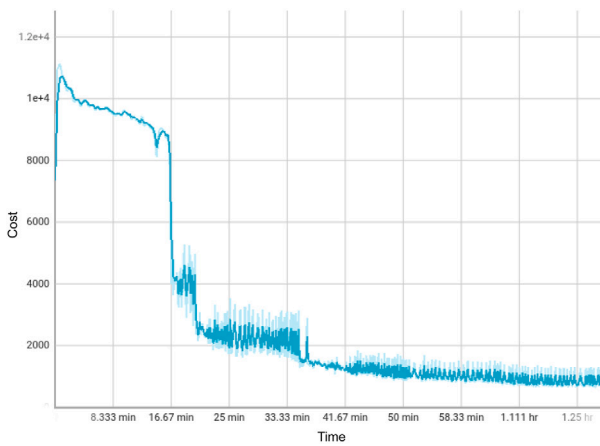
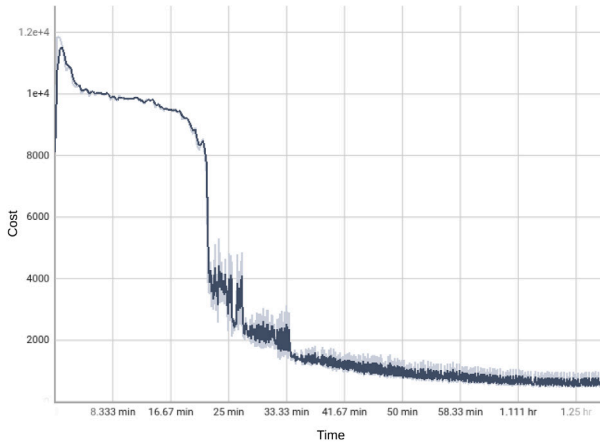
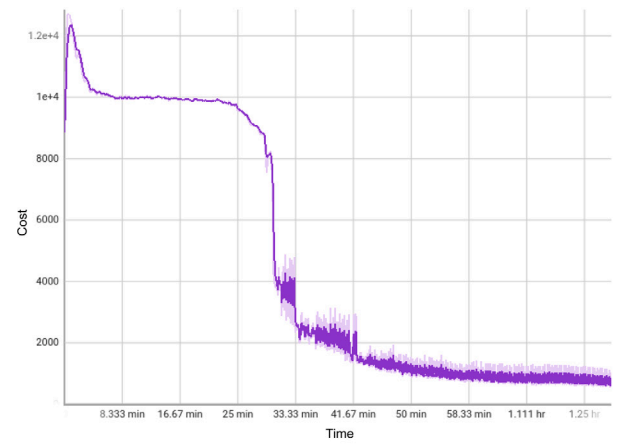
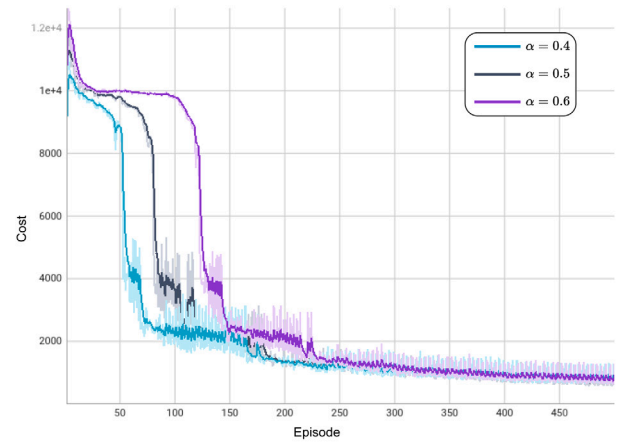


Fig. 8. Convergence time of the PPO strategy.

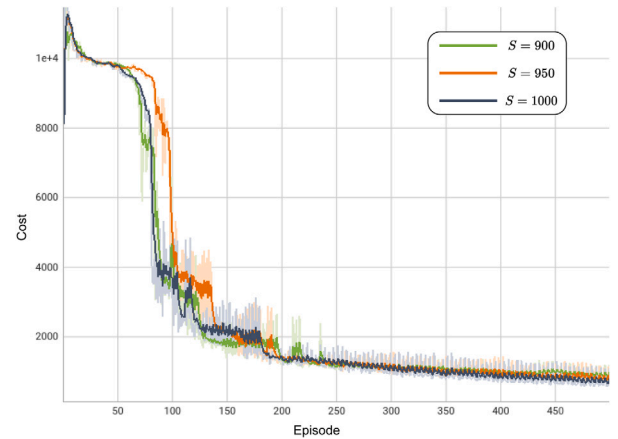
(a) $\alpha = 0.4$ (b) $\alpha = 0.5$ Fig. 9. Convergence time of the PPO strategy under different values of α .

offloading policy without any prior knowledge of the environment. These two strategies are: the DQN strategy with discrete action space, also used as benchmark, and the PPO strategy with continuous action space. From our simulations using a real dataset, we found that the DQN strategy, although it gives good results, is highly sensitive to hyperparameters tuning in the learning phase compared to the PPO strategy, which exhibits stable performance and much better results. Moreover, both DRL algorithms achieve superior performance over other baseline strategies. For instance, the PPO strategy improved the

(c) $\alpha = 0.6$ 

(d) Side by side comparison

Fig. 9. (continued).

Fig. 10. The performance of our PPO algorithm for different values of the parameter S (steps per epoch).

average cost by 12% compared to the DQN and DDQN strategies, as they both converge towards the same minimum cost, and by 37%, 73% compared to the RANDOM and GREEDY strategies respectively. Also, the average cost of the DQN strategy is improved by 21% compared to the RANDOM strategy, and by 52% compared to the GREEDY strategy.

This work is a step towards showcasing the power of DRL algorithms to achieve better offloading policies in the next-generation MEC MIMO-based networks, which is a highly practical issue. Therefore, in the light of the adopted assumptions in our problem formulation, this

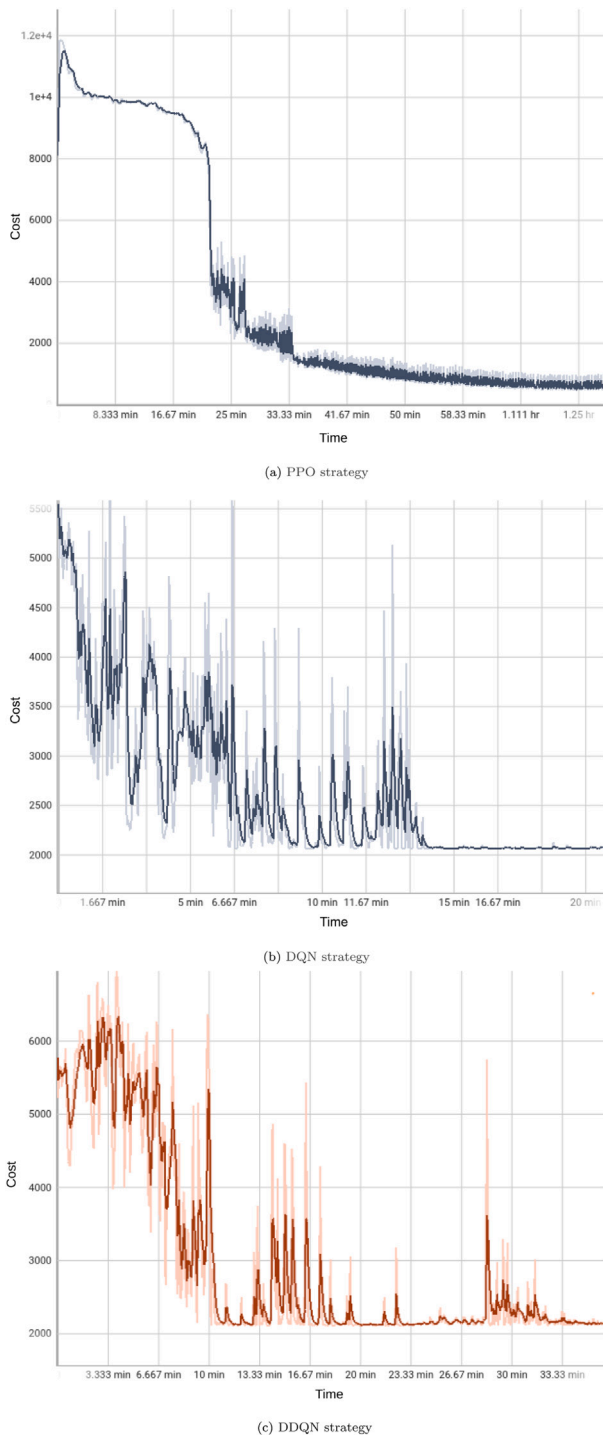


Fig. 11. Training performance of the PPO, DQN and DDQN strategies.

work can be used for practical implementation. However, taking into consideration the complex real-world scenarios, the problem should be formulated to capture all the details such as the MEC power and computation capabilities, the geo-location of each user with regard to the base station, and the different end devices hardware characteristics and the application requirements. This will require much more data and huge computation power for the neural networks to converge. In this perspective, in our future work, we plan to incorporate more environment details such as the computation capacities of the MEC server and a multi-core CPU architecture for the MD devices. We also plan to consider a collaborative multi-user scenario where constrained server resources are shared among them.

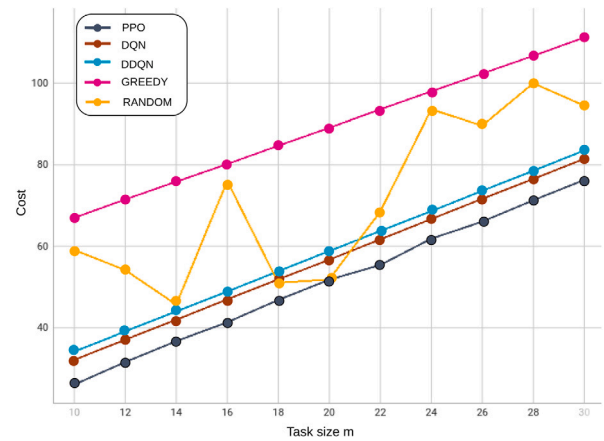


Fig. 12. Performance comparison under different task sizes.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

Jamal Bentahar and Rachida Dssouli are supported by the Natural Sciences and Engineering Research Council of Canada (NSERC). Jamal Bentahar is also supported by the Department of national Defence of Canada (DnD, the IDEaS program) and by Mitacs.

References

- [1] L. Huang, X. Feng, C. Zhang, L. Qian, Y. Wu, Deep reinforcement learning-based joint task offloading and bandwidth allocation for multi-user mobile edge computing, *Digit. Commun. Netw.* 5 (1) (2019) 10–17.
- [2] J. Wang, L. Zhao, J. Liu, N. Kato, Smart resource allocation for mobile edge computing: A deep reinforcement learning approach, *IEEE Trans. Emerg. Top. Comput.* (2019) 1.
- [3] X. Chen, H. Zhang, C. Wu, S. Mao, Y. Ji, M. Bennis, Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning, *IEEE Internet Things J.* 6 (3) (2019) 4005–4018.
- [4] F. Qiao, J. Wu, J. Li, A.K. Bashir, S. Mumtaz, U. Tariq, Trustworthy edge storage orchestration in intelligent transportation systems using reinforcement learning, *IEEE Trans. Intell. Transp. Syst.* 22 (7) (2020) 4443–4456.
- [5] B. Dab, N. Aitsaadi, R. Langar, Q-learning algorithm for joint computation offloading and resource allocation in edge cloud, in: 2019 IFIP/IEEE Symposium on Integrated Network and Service Management (IM), 2019, pp. 45–52.
- [6] T.Q. Dinh, Q.D. La, T.Q. Quek, H. Shin, Learning for computation offloading in mobile edge computing, *IEEE Trans. Commun.* 66 (12) (2018) 6353–6367.
- [7] T. Alfakih, M.M. Hassan, A. Gumaei, C. Savaglio, G. Fortino, Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on SARSA, *IEEE Access* 8 (2020) 54074–54084.
- [8] N. Abbas, Y. Zhang, A. Taherkordi, T. Skeie, Mobile edge computing: A survey, *IEEE Internet Things J.* 5 (1) (2018) 450–465.
- [9] J. Xu, L. Chen, S. Ren, Online learning for offloading and autoscaling in energy harvesting mobile edge computing, 2017, arXiv.
- [10] L. Ale, N. Zhang, X. Fang, X. Chen, S. Wu, L. Li, Delay-aware and energy-efficient computation offloading in mobile edge computing using deep reinforcement learning, *IEEE Trans. Cogn. Commun. Netw.* (2021).
- [11] H. Meng, D. Chao, Q. Guo, Deep reinforcement learning based task offloading algorithm for mobile-edge computing systems, in: Proceedings of the 2019 4th International Conference on Mathematics and Artificial Intelligence, in: ICAI 2019, Association for Computing Machinery, 2019, pp. 90–94.
- [12] J. Cao, W. Feng, N. Ge, J. Lu, Delay characterization of mobile-edge computing for 6G time-sensitive services, *IEEE Internet Things J.* 8 (5) (2021) 3758–3773.

- [13] N. Shlezinger, G.C. Alexandropoulos, M.F. Imani, Y.C. Eldar, D.R. Smith, Dynamic metasurface antennas for 6G extreme massive MIMO communications, *IEEE Wirel. Commun.* 28 (2) (2021) 106–113.
- [14] R. Chataut, R. Akl, Massive MIMO systems for 5G and beyond networks—overview, recent trends, challenges, and future research direction, *Sensors* 20 (10) (2020).
- [15] O. Alamu, B. Iyaomolere, A. Abdulrahman, An overview of massive MIMO localization techniques in wireless cellular networks: Recent advances and outlook, *Ad Hoc Netw.* 111 (2021) 102353.
- [16] M. Zeng, W. Hao, O.A. Dobre, Z. Ding, H.V. Poor, Massive MIMO-assisted mobile edge computing: Exciting possibilities for computation offloading, *IEEE Veh. Technol. Mag.* 15 (2) (2020) 31–38.
- [17] L. Pescosolido, M. Conti, A. Passarella, On the impact of the physical layer model on the performance of D2D-offloading in vehicular environments, *Ad Hoc Netw.* 81 (2018) 197–210.
- [18] S. Zhang, Z. Wang, Z. Zhou, Y. Wang, H. Zhang, G. Zhang, H. Ding, S. Mumtaz, M. Guizani, Blockchain and federated deep reinforcement learning based secure cloud-edge-end collaboration in power IoT, *IEEE Wireless Commun.* 29 (2) (2022) 84–91.
- [19] S. Zhang, H. Gu, K. Chi, L. Huang, K. Yu, S. Mumtaz, DRL-based partial offloading for maximizing sum computation rate of wireless powered mobile edge computing network, *IEEE Trans. Wireless Commun.* (2022) 1, <http://dx.doi.org/10.1109/TWC.2022.3188302>.
- [20] K. Jiang, H. Zhou, D. Li, X. Liu, S. Xu, A Q-learning based method for energy-efficient computation offloading in mobile edge computing, in: 29th International Conference on Computer Communications and Networks, ICCCN 2020, Honolulu, HI, USA, IEEE, 2020, pp. 1–7.
- [21] H. Sami, J. Bentahar, A. Mourad, H. Otrok, E. Damiani, Graph convolutional recurrent networks for reward shaping in reinforcement learning, *Inform. Sci.* 608 (2022) 63–80.
- [22] A. Mazloomi, H. Sami, J. Bentahar, H. Otrok, A. Mourad, Reinforcement learning framework for server placement and workload allocation in multi-access edge computing, *IEEE Internet Things J.* (2022) <http://dx.doi.org/10.1109/JIOT.2022.3205051>.
- [23] M. Taghavi, J. Bentahar, H. Otrok, K. Bakhtiyari, A reinforcement learning model for the reliability of blockchain oracles, *Expert Syst. Appl.* (2022) <http://dx.doi.org/10.1016/j.eswa.2022.119160>.
- [24] X. Chen, H. Zhang, C. Wu, S. Mao, Y. Ji, M. Bennis, Performance optimization in mobile-edge computing via deep reinforcement learning, in: 88th IEEE Vehicular Technology Conference, VTC Fall 2018, Chicago, IL, USA, August 27–30, 2018, pp. 1–6.
- [25] R. Zhao, X. Wang, J. Xia, L. Fan, Deep reinforcement learning based mobile edge computing for intelligent Internet of Things, *Phys. Commun.* 43 (2020) 101184.
- [26] S. Deng, H. Zhao, W. Fang, J. Yin, S. Dustdar, A.Y. Zomaya, Edge intelligence: The confluence of edge computing and artificial intelligence, *IEEE Internet Things J.* 7 (8) (2020) 7457–7469.
- [27] Y. Wang, M. Sheng, X. Wang, L. Wang, J. Li, Mobile-edge computing: Partial computation offloading using dynamic voltage scaling, *IEEE Trans. Commun.* 64 (10) (2016) 4268–4282.
- [28] F. Messaoudi, A. Ksentini, P. Bertin, On using edge computing for computation offloading in mobile network, in: IEEE Global Communications Conference, GLOBECOM 2017, Singapore, December 4–8, 2017, pp. 1–7.
- [29] X. Chen, L. Jiao, W. Li, Efficient multi-user computation offloading for mobile-edge cloud computing, *IEEE/ACM Trans. Netw.* 24 (5) (2016) 2795–2808.
- [30] M. Chen, Y. Hao, Task offloading for mobile edge computing in software defined ultra-dense network, *IEEE J. Sel. Areas Commun.* 36 (3) (2018) 587–597.
- [31] M.E. Khoda, M.A. Razzaque, A. Almogren, M.M. Hassan, A. Alamri, A. Alelaiwi, Efficient computation offloading decision in mobile cloud computing over 5G network, *Mob. Netw. Appl.* 21 (5) (2016) 777–792.
- [32] M. Zeng, M. Zeng, W. Hao, O.A. Dobre, H.V. Poor, Delay minimization for massive MIMO assisted mobile edge computing, *IEEE Trans. Veh. Technol.* 69 (6) (2020) 6788–6792.
- [33] Y. Hao, Q. Ni, H. Li, S. Hou, Energy-efficient multi-user mobile-edge computation offloading in massive MIMO enabled HetNets, in: IEEE International Conference on Communications, ICC, Shanghai, China, May 20–24, IEEE, 2019, pp. 1–6.
- [34] C. Ding, J. bo Wang, H. Zhang, M. Lin, J. Wang, Joint MU-MIMO precoding and resource allocation for mobile-edge computing, *IEEE Trans. Wireless Commun.* 20 (2021) 1639–1654.
- [35] H. Sami, A. Mourad, Dynamic on-demand fog formation offering on-the-fly IoT service deployment, *IEEE Trans. Netw. Serv. Manag.* 17 (2) (2020) 1026–1039.
- [36] H. Sami, A. Mourad, H. Otrok, J. Bentahar, Demand-driven deep reinforcement learning for scalable fog and service placement, *IEEE Trans. Serv. Comput.* (2021).
- [37] H. Sami, H. Otrok, J. Bentahar, A. Mourad, AI-based resource provisioning of IoE services in 6G: A deep reinforcement learning approach, *IEEE Trans. Netw. Serv. Manag.* 18 (3) (2021) 3527–3540.
- [38] <https://www.mathworks.com/discovery/massive-mimo.html>.
- [39] L. Lu, G.Y. Li, A.L. Swindlehurst, A. Ashikhmin, R. Zhang, An overview of massive MIMO: Benefits and challenges, *IEEE J. Sel. Top. Signal Process.* 8 (5) (2014) 742–758.
- [40] S. Noh, M.D. Zoltowski, Y. Sung, D.J. Love, Pilot beam pattern design for channel estimation in massive MIMO systems, *IEEE J. Sel. Top. Signal Process.* 8 (5) (2014) 787–801.
- [41] S. Noh, J. Song, Y. Sung, Fast beam search and refinement for millimeter-wave massive MIMO based on two-level phased arrays, *IEEE Trans. Wireless Commun.* 19 (10) (2020) 6737–6751.
- [42] S. Nath, Y. Li, J. Wu, P. Fan, Multi-user multi-channel computation offloading and resource allocation for mobile edge computing, in: IEEE International Conference on Communications, ICC, Dublin, Ireland, June 7–11, 2020, pp. 1–6.
- [43] T. Ouyang, Z. Zhou, X. Chen, Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing, *IEEE J. Sel. Areas Commun.* 36 (10) (2018) 2333–2345.
- [44] Y. Yu, T. Wang, S.C. Liew, Deep-reinforcement learning multiple access for heterogeneous wireless networks, *IEEE J. Sel. Areas Commun.* 37 (6) (2019) 1277–1290.
- [45] W. Zhang, Y. Wen, K. Guan, D. Kilper, H. Luo, D.O. Wu, Energy-optimal mobile cloud computing under stochastic wireless channel, *IEEE Trans. Wireless Commun.* 12 (9) (2013) 4569–4581.
- [46] X. Hu, K.-K. Wong, K. Yang, Wireless powered cooperation-assisted mobile edge computing, *IEEE Trans. Wireless Commun.* 17 (4) (2018) 2375–2388.
- [47] Z. Ding, P. Fan, H.V. Poor, Impact of Non-Orthogonal Multiple Access on the Offloading of Mobile Edge Computing, *IEEE Trans. Commun.* 67 (1) (2018) 375–390.
- [48] Y. Mao, J. Zhang, K.B. Letaief, Dynamic computation offloading for mobile-edge computing with energy harvesting devices, *IEEE J. Sel. Areas Commun.* 34 (12) (2016) 3590–3605.
- [49] G. Rjoub, O.A. Wahab, J. Bentahar, A. Bataineh, Trust-driven reinforcement selection strategy for federated learning on IoT devices, *Computing* (2022) <http://dx.doi.org/10.1007/s00607-022-01078-1>.
- [50] G. Rjoub, O.A. Wahab, J. Bentahar, R. Cohen, A. Bataineh, Trust-augmented deep reinforcement learning for federated learning client selection, *Inf. Syst. Front.* (2022) <http://dx.doi.org/10.1007/s10796-022-10307-z>.
- [51] G. Rjoub, J. Bentahar, O.A. Wahab, A.S. Bataineh, Deep and reinforcement learning for automated task scheduling in large-scale cloud computing systems, *Concurr. Comput. Pract. Exp.* 33 (23) (2021).
- [52] R.S. Sutton, F. Bach, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press Ltd, 2018.
- [53] V. François-Lavet, P. Henderson, R. Islam, M.G. Bellemare, J. Pineau, An introduction to deep reinforcement learning, *Found. Trends Mach. Learn.* 11 (3–4) (2018) 219–354.
- [54] S. Wang, M. Chen, X. Liu, C. Yin, S. Cui, H.V. Poor, A machine learning approach for task and resource allocation in mobile-edge computing-based networks, *IEEE Internet Things J.* 8 (3) (2021) 1358–1372.
- [55] L.P. Kaelbling, M.L. Littman, A.W. Moore, Reinforcement learning: A survey, *J. Artificial Intelligence Res.* 4 (1996) 237–285.
- [56] X. Xu, L. Zuo, Z. Huang, Reinforcement learning algorithms with function approximation: Recent advances and applications, *Inform. Sci.* 261 (2014) 1–31.
- [57] V. François-Lavet, R. Fonteneau, D. Ernst, How to discount deep reinforcement learning: Towards new dynamic strategies, 2015, CoRR abs/1512.02011. URL <http://arxiv.org/abs/1512.02011>.
- [58] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M.A. Riedmiller, Playing Atari with deep reinforcement learning, 2013, CoRR arXiv:1312.5602. URL <http://arxiv.org/abs/1312.5602>.
- [59] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, CoRR arXiv:1707.06347. URL <http://arxiv.org/abs/1707.06347>.
- [60] A. Alagha, S. Singh, R. Mizouni, J. Bentahar, H. Otrok, Target localization using multi-agent deep reinforcement learning with proximal policy optimization, *Future Gener. Comput. Syst.* 136 (2022) 342–357.
- [61] J. Schulman, P. Moritz, S. Levine, M.I. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation, in: Y. Bengio, Y. LeCun (Eds.), 4th International Conference on Learning Representations, ICLR, San Juan, Puerto Rico, May 2–4, Conference Track Proceedings, 2016, Poster.
- [62] Massive mimo channel state information measurements, 2020, URL https://homes.esat.kuleuven.be/~sdebast/measurements/measurements_index.html.
- [63] H. van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double Q-learning, in: D. Schuurmans, M.P. Wellman (Eds.), Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, February 12–17, Phoenix, Arizona, USA, AAAI Press, 2016, pp. 2094–2100.



Abdeladim Sadiki received the bachelor's degree in engineering and information technology from the National Institute of Posts and Telecommunications, Morocco. He obtained his MASC. degree in Quality Systems Engineering from Concordia University, Institute of information Systems Engineering, Canada in 2022. His research interests include software engineering, machine learning, services computing and embedded cyber-physical systems.



Jamal Bentahar received the Ph.D. degree in computer science and software engineering from Laval University, Canada, in 2005. He is a Professor with Concordia Institute for Information Systems Engineering, Concordia University, Canada. From 2005 to 2006, he was a Postdoctoral Fellow with Laval University, and then NSERC Postdoctoral Fellow at Simon Fraser University, Canada. He was an NSERC Co-Chair for Discovery Grants for Computer Science (2016–2018). His research interests include computational logics, model checking, reinforcement and deep learning, multi-agent systems, and services computing.



Rachida Dssouli is a professor and the founding of Concordia Institute for Information Systems Engineering at Concordia University, Canada. She earned her Ph.D. degree in Computer Science (1987) from (DIRO) University of Montreal, Canada. Her research interests are in software engineering, communication software engineering, requirements engineering, systems engineering, services engineering, and formal methods. She applied her research to telecommunication, eHealth and avionics software. She published more than 230-refereed papers in journals and



Abdeslam En-Nouaary received the Ph.D. degree in computer science from the University of Montreal in 2001. He is currently a Full Professor at INPT, Rabat, Morocco. Before joining INPT, he was an Associate Professor at Concordia University, Montreal, Canada, from 2001 to 2008. His main research interests are modeling, validation, and prototyping of distributed, real-time, and embedded systems.



Hadi Otrouk received his Ph.D. in ECE from Concordia University. He holds an Associate Professor position in the department of Electrical Engineering and Computer Science (EECS) at Khalifa University. Also, he is an Affiliate Associate Professor in the Concordia Institute for Information Systems Engineering at Concordia University, Montreal, Canada, and an Affiliate Associate Professor in the Electrical department at Ecole de Technologie Supérieure (ETS), Montreal, Canada. His research interests include the domain of blockchain, reinforcement learning, crowd sensing and sourcing, ad hoc networks, and cloud security. He cochaired several committees at various IEEE conferences. He is also an Associate Editor at IEEE Transactions on Network and Service Management (TNSM), Ad-hoc networks (Elsevier), and IEEE Network. He also served from 2015 to 2019 as an Associate Editor at IEEE Communications Letters.